

UNIVERSIDAD POLITÉCNICA DE CHIAPAS
MAESTRÍA EN CIENCIAS E INNOVACIÓN TECNOLÓGICA

ASIGNATURA
LÓGICA Y PROGRAMACIÓN

EVALUACIÓN 04 - Informe Individual

ALUMNO
GUILLERMO TINOCO RAMOS

DOCENTE
DR. HORACIO IRÁN SOLÍS CISNEROS

Tuxtla Gutiérrez, Chiapas
12 de marzo de 2026

1. Introducción

El desarrollo de sistemas concurrentes y paralelos constituye una estrategia fundamental para mejorar el rendimiento de aplicaciones computacionales modernas. La paralelización permite aprovechar múltiples unidades de procesamiento para reducir el tiempo total de ejecución de un programa.

En este informe se analiza el impacto de la concurrencia en el rendimiento de un pipeline de procesamiento mediante la comparación entre una implementación secuencial y una implementación concurrente. Para ello se realiza un experimento de benchmarking que evalúa el comportamiento del sistema bajo diferentes tamaños de carga de trabajo.

El análisis se centra en métricas clásicas de evaluación de sistemas paralelos como el tiempo de ejecución, el speedup y la eficiencia. Asimismo, se interpreta el comportamiento observado a la luz de modelos teóricos de paralelización, particularmente la Ley de Amdahl.

2. Sistema Experimental

Los experimentos se ejecutaron en el siguiente entorno computacional:

Hardware

- Equipo: MacBook Pro
- Procesador: Apple M3 Pro
- Núcleos totales: 12 (6 Performance + 6 Efficiency)
- Memoria RAM: 18 GB

Software

- Sistema operativo: macOS
- Lenguaje de programación: Python
- Ejecución desde terminal del sistema

Entorno de ejecución

Los experimentos fueron ejecutados mediante un script automatizado denominado `compare_P_vs_Q.py`, el cual ejecuta distintas configuraciones del sistema y registra automáticamente los tiempos de ejecución.

El sistema concurrente utiliza múltiples *workers* para ejecutar tareas en paralelo. En los experimentos se utilizó una configuración de cuatro workers.

Los experimentos se ejecutaron desde la terminal del sistema operativo con el objetivo de reducir interferencias externas y mantener condiciones de ejecución consistentes entre las distintas corridas experimentales.

Carga de trabajo

La carga de trabajo se controla mediante dos parámetros principales:

- Tamaño del problema n
- Trabajo computacional artificial definido por el parámetro `cpu_work`

El parámetro `cpu_work` introduce una carga de trabajo artificial de tipo CPU-bound que simula operaciones computacionales intensivas. Este enfoque permite controlar de manera reproducible la cantidad de trabajo ejecutado por cada tarea del pipeline y evaluar el comportamiento del sistema bajo diferentes niveles de carga computacional.

Los tamaños de problema evaluados fueron:

$$n \in \{10, 25, 50, 100, 200\}$$

Cada ejecución del experimento se repite múltiples veces (`cycles = 30`) con el fin de reducir la variabilidad introducida por el sistema operativo, el intérprete de Python y la planificación de procesos.

Durante cada ejecución se registran estadísticas adicionales como el tiempo mínimo, máximo y la desviación estándar, lo cual permite evaluar la estabilidad de las mediciones experimentales.

3. Metodología de Benchmarking

Con el objetivo de evaluar el impacto de la concurrencia en el desempeño del sistema, se compararon dos configuraciones principales:

- **Estado P:** implementación secuencial del sistema.
- **Estado Q:** implementación concurrente basada en paralelización mediante múltiples workers.

Para cada valor de n se ejecutaron tres configuraciones:

1. Estado P (secuencial)
2. Estado Q con concurrencia mínima (`async`, 1 worker)
3. Estado Q con paralelización completa (4 workers)

Esta metodología permite separar dos efectos distintos en el análisis de rendimiento:

- El impacto del rediseño arquitectónico del pipeline.
- El impacto del paralelismo al incrementar el número de workers.

Los resultados de cada ejecución se almacenan en un archivo CSV con el siguiente formato:

```
n,t_p,t_q_async,t_q_full,speedup_async,speedup_full,efficiency
```

4. Speedup

El *speedup* mide la mejora en tiempo de ejecución al comparar una implementación paralela con una secuencial.

$$S = \frac{T_P}{T_Q}$$

donde:

- T_P es el tiempo de ejecución secuencial
- T_Q es el tiempo de ejecución concurrente

En este trabajo se distinguen dos tipos de speedup:

- **Speedup arquitectónico:** mejora obtenida al comparar la implementación secuencial con la arquitectura concurrente utilizando un único worker.
- **Speedup total:** mejora obtenida al comparar la implementación secuencial con la versión concurrente completamente paralelizada.

5. Eficiencia

La eficiencia paralela mide el grado de aprovechamiento de los recursos de procesamiento disponibles al ejecutar una versión paralela de un algoritmo.

En este trabajo la eficiencia se calcula considerando únicamente la mejora obtenida al incrementar el número de workers dentro de la misma arquitectura concurrente:

$$E = \frac{S_{par}}{p}$$

donde:

- $S_{par} = \frac{T_{Q_async}}{T_{Q_full}}$
- p es el número de workers

Esta formulación permite medir el grado real de aprovechamiento del paralelismo sin mezclar los efectos de optimización arquitectónica con los efectos del paralelismo.

En el contexto del experimento presentado en este trabajo, la eficiencia se utiliza para evaluar qué tan bien el sistema concurrente aprovecha los cuatro workers disponibles al ejecutar el pipeline de procesamiento. Valores de eficiencia cercanos a 1 indicarían un aprovechamiento casi ideal del paralelismo, mientras que valores menores reflejan overhead de coordinación o secciones secuenciales del sistema.

6. Ley de Amdahl

La Ley de Amdahl establece el límite teórico del speedup alcanzable mediante paralelización:

$$S_{max} = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

donde α representa la fracción paralelizable del programa.

Esta ley permite estimar qué porcentaje del sistema puede beneficiarse de la ejecución en paralelo y cuál es el límite teórico del rendimiento que puede alcanzarse al incrementar el número de workers.

En este trabajo, la Ley de Amdahl se utiliza para interpretar los resultados experimentales del benchmark y estimar la fracción paralelizable del pipeline de procesamiento evaluado. A partir del speedup observado al comparar la versión concurrente con un worker y la versión con cuatro workers, es posible aproximar el valor de α y analizar qué tan cercano se encuentra el sistema al límite teórico de paralelización.

7. Resultados Experimentales

n	T_P	T_{Q_async}	T_{Q_full}	S_{arch}	S_{total}	E
10	0.568	0.086	0.056	6.60	10.14	0.38
25	1.421	0.158	0.075	8.99	18.95	0.53
50	2.831	0.265	0.106	10.68	26.71	0.63
100	5.926	0.488	0.170	12.14	34.86	0.72
200	12.153	0.926	0.295	13.12	41.20	0.78

Cuadro 1: Resultados experimentales del benchmark

8. Interpretación de Resultados

Los resultados experimentales muestran una reducción significativa en el tiempo de ejecución al utilizar la arquitectura concurrente.

Al comparar la implementación secuencial con la arquitectura concurrente utilizando un único worker, se observa un speedup de entre 6.6 y 13.1. Este comportamiento indica que una parte importante de la mejora se debe al rediseño arquitectónico del pipeline.

Este resultado evidencia que el rendimiento del sistema no depende únicamente del paralelismo, sino también de la forma en que la arquitectura del software organiza el flujo de procesamiento. En particular, la reorganización del pipeline reduce overheads presentes en la implementación secuencial original y permite una ejecución más eficiente incluso antes de habilitar paralelismo real.

Posteriormente, al habilitar paralelismo mediante múltiples workers, se obtiene una mejora adicional atribuible al paralelismo. Cuando se compara la versión concurrente con un único

worker contra la versión con cuatro workers, el speedup paralelo observado alcanza valores cercanos a 3.14. Este resultado es consistente con el número de workers utilizados y refleja el efecto directo de la paralelización del trabajo.

Es importante notar que el speedup total observado en la tabla alcanza valores superiores a 40x para los tamaños de problema mayores. Sin embargo, este valor no debe interpretarse únicamente como efecto del paralelismo. En realidad, dicho speedup es el resultado combinado de dos factores distintos:

- Mejora arquitectónica del pipeline respecto a la implementación secuencial.
- Paralelización del trabajo mediante múltiples workers.

Al separar ambos efectos se observa que la mayor parte de la mejora inicial proviene del rediseño arquitectónico del sistema, mientras que el paralelismo aporta una aceleración adicional cercana al límite esperado para cuatro workers.

Asimismo, la eficiencia paralela aumenta conforme crece el tamaño del problema, pasando de aproximadamente 0.38 para cargas pequeñas hasta cerca de 0.78 para el mayor tamaño evaluado. Este comportamiento indica que el overhead de coordinación entre workers se amortiza mejor cuando la carga computacional es mayor.

9. Estimación de la fracción paralelizable

Aplicando la Ley de Amdahl al mayor tamaño del problema evaluado:

$$S_{par} \approx 3,14, \quad p = 4$$

se obtiene una estimación aproximada de:

$$\alpha \approx 0,90$$

Esto sugiere que aproximadamente el 90 % del trabajo realizado por el sistema es paralelizable, mientras que el 10 % restante corresponde a secciones secuenciales o a overhead de coordinación.

10. Limitaciones y Amenazas a la Validez

Los experimentos fueron ejecutados en una sola máquina, por lo que los resultados pueden depender de las características específicas del hardware utilizado.

La carga de trabajo utilizada corresponde a una simulación controlada mediante el parámetro `cpu_work`, lo cual podría diferir de escenarios reales con patrones de acceso a memoria más complejos.

Asimismo, aunque se realizaron múltiples repeticiones de los experimentos, factores externos como procesos del sistema operativo pueden introducir variabilidad en las mediciones.

11. Conclusiones

El experimento demuestra que la arquitectura concurrente propuesta mejora significativamente el rendimiento respecto a la implementación secuencial original.

Los resultados muestran que una fracción importante de la mejora inicial se debe al rediseño arquitectónico del pipeline. Incluso utilizando un único worker, la arquitectura concurrente logra speedups superiores a 10x en comparación con la versión secuencial original.

Posteriormente, al habilitar paralelismo mediante múltiples workers, se obtiene una mejora adicional atribuible al paralelismo. Cuando se analiza exclusivamente el efecto del paralelismo dentro de la misma arquitectura concurrente, el speedup observado es cercano a 3.14 al utilizar cuatro workers.

La eficiencia paralela obtenida alcanza valores cercanos a 0.78 para los tamaños de problema mayores, lo cual indica un buen aprovechamiento de los recursos computacionales disponibles.

Finalmente, el análisis basado en la Ley de Amdahl sugiere que aproximadamente el 90 % del trabajo realizado por el sistema es paralelizable. En conjunto, los resultados muestran que las mejoras en rendimiento observadas no se deben exclusivamente al paralelismo, sino también al rediseño arquitectónico del pipeline, lo cual resalta la importancia de considerar tanto la arquitectura del software como las estrategias de paralelización al diseñar sistemas de alto rendimiento.